TRADEFOLIO

Full-Stack Microservices Portfolio Analytics & Trade Notification Platform

Technical Project Specification & Implementation Blueprint

Microservices • Docker • Distributed Systems • .NET • PostgreSQL • RabbitMQ • Redis • React • Cloud • DevOps

Property	Definition
Target duration	6 weeks
Primary roles	Backend / SDE, DevOps / Cloud-leaning SDE, Data Analyst / Data Engineer
Primary goal	Demonstrate a production-style, containerized full-stack microservices system
Business capability	Trades, holdings, portfolio analytics, market data and notifications
MVP principle	Complete vertical slice first; advanced cloud/BI/observability after correctness

Source of truth: the TradeFolio project plan supplied by the user, with the architecture centered on microservices, Docker and full-stack implementation.

1. Executive Summary

TradeFolio is a full-stack portfolio management and analytics platform designed to demonstrate real-world backend and distributed-systems engineering rather than simple CRUD. Users authenticate, record buy/sell trades, maintain consistent holdings, receive trade notifications, and view portfolio analytics through a React dashboard and a BI layer.

The architectural centerpiece is a set of independently deployable ASP.NET Core services connected through REST APIs and RabbitMQ events, backed by PostgreSQL and Redis and packaged with Docker. The system explicitly demonstrates transactional correctness, concurrency control, idempotent event processing, caching, automated testing, CI/CD, infrastructure as code, cloud deployment and observability.

Analytics remains a major product feature. It sits on top of reliable trade and portfolio state and gives the distributed architecture a concrete user-facing purpose.

Core project narrative

Trade execution correctness → transactional portfolio consistency → event-driven notifications → market-data caching → portfolio analytics → React/BI experience → containerized deployment → automated testing and CI/CD → cloud infrastructure and observability.

2. Problem, Goals & Boundaries

Retail investors often track trades and holdings across disconnected applications and have limited visibility into actual portfolio performance. TradeFolio centralizes trade records and holdings, maintains consistent portfolio state, retrieves market prices, calculates analytics, and presents personalized insights.

Goals

- Build a complete full-stack product from database to React UI.

- Implement meaningful microservice boundaries.
- Demonstrate transaction and concurrency correctness, especially prevention of overselling.
- Implement FIFO portfolio accounting across partial sells.
- Use RabbitMQ for asynchronous trade-completion events.
- Make Notification Service independently deployable and idempotent.
- Use Redis for market-price caching and define failure behavior.
- Provide P&L, allocation, sector exposure, trade frequency and best/worst performers.
- Provide React and Power BI/Tableau views of the same data.
- Containerize the system with one-command local startup.
- Automate tests, CI/CD and provide an AWS/Terraform deployment path.

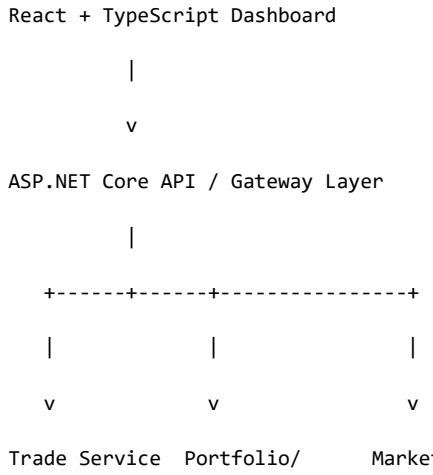
Non-goals

- Real brokerage execution.
- Financial advice or guaranteed returns.
- High-frequency trading infrastructure.
- Complex derivatives accounting in MVP.
- Splitting every calculation into its own microservice.
- Claiming production scale/performance without measurements.

3. Architecture Principles

Principle	Implementation rule
Clear ownership	Each service owns a coherent capability and its write responsibilities.
Correctness first	Trade and holding mutations commit transactionally before asynchronous side effects.
Event-driven integration	Trade completion is propagated through RabbitMQ.
Idempotency	Consumers tolerate redelivery without duplicate side effects.
Container-first	Services, PostgreSQL, Redis and RabbitMQ run in Docker Compose.
API-first UI	React consumes APIs and never accesses databases directly.
Observable	Important flows expose health, logs, metrics and traces.
Secure by default	Hashed passwords, JWT, authorization, validation and externalized secrets.
Measured	Concurrency, query performance, caching and tests are demonstrated reproducibly.
Avoid microservice theater	Use meaningful service boundaries only.

High-level architecture





4. Microservice Boundaries

Service	Owns / responsibility	MVP
Trade Service	Trade execution, validation, transaction boundaries, trade records, outbox events	Yes
Portfolio Service	Holdings, FIFO lots, portfolio state and cost-basis inputs	Yes
Analytics Service	P&L, allocation, sector exposure, frequency, performers, insights	Yes
Market Data Service	External price provider adapter, Redis cache, TTL/failure policy	Yes
Notification Service	RabbitMQ consumer, delivery state, retries, idempotency, notification provider	Yes
Auth/API Layer	Registration/login, JWT, authorization, API versioning, rate limiting/routing	Yes

Keep the number of services small enough to complete in six weeks. These boundaries demonstrate microservices without turning every function into a separate deployment.

5. Functional Requirements

ID	Requirement	Acceptance behavior
FR-01	Registration	Validated user registration; passwords never stored in plaintext.
FR-02	Authentication	Login returns JWT access token.
FR-03	User isolation	Users can access only their own trades, holdings, analytics and notifications.
FR-04	Trades	Create/read/list trade records with validation and controlled mutation

		semantics.
FR-05	Validation	Reject invalid symbols/instruments, non-positive quantities/prices and invalid sides.
FR-06	No overselling	SELL cannot exceed available quantity, including under concurrent requests.
FR-07	Atomicity	Trade and portfolio state changes commit atomically.
FR-08	FIFO	Partial sells consume open buy lots in FIFO order.
FR-09	Events	Successful trade completion produces TradeCompleted event.
FR-10	Notifications	Notification service consumes trade events and creates a notification.
FR-11	Idempotency	Repeated event delivery does not duplicate notification side effects.
FR-12	Retry/DLQ	Transient failures retry; exhausted messages can enter a DLQ.
FR-13	Market data	Retrieve delayed/live prices through a free provider adapter.
FR-14	Redis	Cache market prices with TTL and defined failure behavior.
FR-15	Analytics	P&L, allocation, sector exposure, trade frequency and performers.
FR-16	Insights	Generate deterministic personalized insights from portfolio analytics.
FR-17	Dashboard	React UI shows portfolio, analytics, trades and notifications.
FR-18	BI	Power BI or Tableau consumes the same underlying data/analytics.
FR-19	Pagination	Collection endpoints support pagination and stable ordering.
FR-20	Errors	Consistent ProblemDetails-style error responses.
FR-21	Rate limiting	Token-bucket rate limiting or equivalent.
FR-22	Versioning	Public APIs begin at /api/v1.

6. Domain Model & Invariants

Entity	Purpose	Important fields
User	Authenticated owner	Id, email, password_hash, created_at, status
Instrument	Tradable reference	Id, symbol, name, sector, currency
Trade	Executed transaction	Id, user_id, instrument_id, side, quantity, price, executed_at, idempotency_key
Holding	Current position aggregate	Id, user_id, instrument_id, quantity, updated_at
HoldingLot	Open FIFO buy lot	Id, holding_id, original_quantity, remaining_quantity, purchase_price, purchased_at
MarketPrice	Observed/cached price	instrument_id, price, observed_at, source
NotificationDelivery	Notification state	event_id, user_id, status, attempts, last_error
OutboxMessage	Reliable event publication	event_id, event_type, payload, created_at, published_at, attempts

Business invariants

- Trade quantity and price are positive.
- SELL never exceeds available quantity.

- Successful trades never leave negative holdings.
- Trade and portfolio mutations commit atomically.
- One current Holding exists per user/instrument pair.
- FIFO lots have non-negative remaining quantities.
- event_id is unique.
- Notification processing is idempotent on event_id.
- Every user-owned resource enforces ownership.

7. Trade Execution, Transactions & Concurrency

This is the key backend correctness demonstration. The project must contain a reproducible test proving that two simultaneous sells cannot spend the same shares.

Recommended sell algorithm

1. Begin a PostgreSQL transaction.
2. Validate the authenticated user and Idempotency-Key.
3. Locate and lock the user's Holding for the instrument.
4. Re-read current quantity after acquiring the lock.
5. Reject if requested quantity exceeds available quantity.
6. Read open lots ordered by purchase time ascending.
7. Consume FIFO lots and calculate realized cost basis/P&L.
8. Update/delete affected lots and the aggregate holding.
9. Insert the Trade record.
10. Insert a TradeCompleted event into the transactional outbox.
11. Commit.
12. A background publisher later sends the event to RabbitMQ.

Concurrency test

```
Initial holding: 10 shares

Request A: SELL 7

Request B: SELL 7



Expected:

- exactly one succeeds

- exactly one fails with insufficient quantity

- final holding = 3

- no negative quantity

- one successful trade for the two competing sells

- one corresponding TradeCompleted event
```

Use multiple tasks, a synchronization barrier, a real PostgreSQL Testcontainer and the actual application transaction path. Do not mock the database.

Isolation decision

Preferred MVP: PostgreSQL Read Committed plus explicit row-level locking for the holding being mutated. Serializable can be evaluated as an alternative, but the project should document the chosen strategy and the anomalies it prevents.

8. Portfolio Accounting & DSA

FIFO is recommended because it creates a concrete algorithmic problem in addition to SQL aggregation.

```
BUY  10 @ 100

BUY   5 @ 120

SELL 12 @ 150


FIFO cost basis = 10*100 + 2*120 = 1240

Sale proceeds = 12*150 = 1800

Realized P&L = 560

Remaining lot = 3 @ 120
```

For trade frequency, demonstrate a sliding-window algorithm for a selected interval or the equivalent SQL/window-function approach, including complexity reasoning.

9. PostgreSQL Database Design

Use a normalized schema around users, instruments, trades, holdings/lots, notifications and outbox messages.

Index	Reason
Users(email)	Fast login/identity lookup and uniqueness.
Trades(user_id, executed_at DESC)	Trade history and recent activity.
Trades(user_id, instrument_id, executed_at)	Instrument-specific analytics.
Holdings(user_id, instrument_id)	Fast portfolio lookup and uniqueness.
HoldingLots(holding_id, purchased_at)	FIFO traversal.
OutboxMessages(published_at, created_at)	Efficient pending-event scans.
NotificationDelivery(event_id)	Fast idempotency lookup.
MarketPrice(instrument_id, observed_at DESC)	Latest-price lookup.

Document at least two important queries with EXPLAIN/EXPLAIN ANALYZE and explain the relevant indexes.

10. REST API

Method	Endpoint	Purpose
POST	/api/v1/auth/register	Register
POST	/api/v1/auth/login	Authenticate / JWT
GET	/api/v1/users/me	Current profile
GET	/api/v1/holdings	Current holdings
GET	/api/v1/trades?page=1&pageSize=20	Paginated trades
POST	/api/v1/trades	Execute trade
GET	/api/v1/trades/{id}	Trade details
GET	/api/v1/analytics/summary	Portfolio KPIs
GET	/api/v1/analytics/pnl	P&L
GET	/api/v1/analytics/allocation	Allocation

GET	/api/v1/analytics/sectors	Sector exposure
GET	/api/v1/analytics/frequency	Trade frequency
GET	/api/v1/analytics/performers	Best/worst performers
GET	/api/v1/market-prices/{symbol}	Market price
GET	/api/v1/notifications	Notifications

Standard error response

```
{
  "type": "https://api.tradefolio.local/errors/validation",
  "title": "Validation failed",
  "status": 400,
  "traceId": "01J...",
  "errors": {"quantity": ["Quantity must be greater than zero."]}
}
```

- Use /api/v1.
- Use consistent HTTP status codes and ProblemDetails.
- Paginate collection endpoints.
- Return correlation/trace IDs.
- Document with Swagger/OpenAPI.
- Apply token-bucket rate limiting.
- Use Idempotency-Key for trade creation.

11. RabbitMQ & Event-Driven Architecture

TradeCompleted event

```
{
  "eventId": "uuid",
  "eventType": "TradeCompleted",
  "schemaVersion": 1,
  "tradeId": "uuid",
  "userId": "uuid",
  "instrumentId": "uuid",
  "symbol": "AAPL",
  "side": "SELL",
  "quantity": 12,
  "price": 150.00,
  "executedAt": "2026-10-16T18:30:00Z",
  "correlationId": "uuid"
}
```

Transactional outbox

The Trade Service writes the trade, portfolio changes and outbox event in one database transaction. A background publisher reads unpublished outbox rows and publishes them to RabbitMQ. This prevents a committed trade from being lost merely because RabbitMQ is unavailable at that moment.

Notification idempotency

- Persist event_id processing state.
- If an event was already successfully processed, acknowledge without sending again.
- Track attempts and failure reason.
- Use exponential backoff for transient failures.
- Route exhausted messages to a DLQ.
- Keep notification provider behind an interface so local development can use a mock provider.

If RabbitMQ is down, a successful trade remains committed and the outbox publisher retries after the broker returns.

12. Market Data & Redis

- Market Data Service owns the external provider adapter.
- Redis caches prices with a defined TTL.
- Cache hits avoid external calls.
- Cache misses populate Redis after provider success.
- Provider errors/rate limits must not take down trade execution.
- A stale value may be returned only when explicitly marked stale.
- Market-price retrieval never mutates trade history.

Choose the free provider at implementation time based on availability, API limits and terms. Keep the service vendor-neutral through an adapter.

13. Analytics Requirements

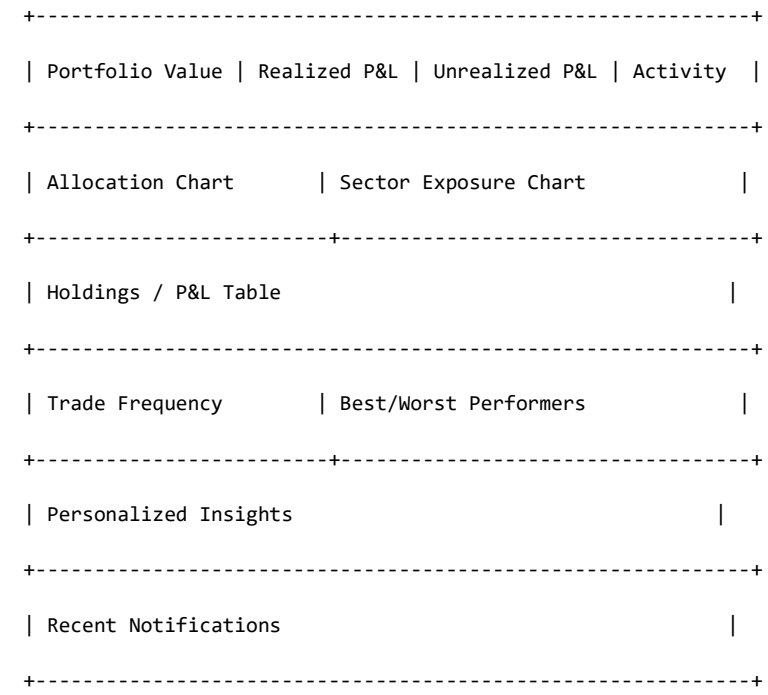
Analytics	Definition
Portfolio value	Sum of holding quantity × latest market price.
Realized P&L	Sell proceeds minus FIFO cost basis consumed by sells.
Unrealized P&L	Current market value of open lots minus remaining cost basis.
Allocation	Instrument/asset market value divided by total portfolio value.
Sector exposure	Market value grouped by sector.
Trade frequency	Trade count over selected windows using sliding-window or SQL window technique.
Best performers	Rank instruments by selected performance/P&L measure.
Worst performers	Rank instruments by selected performance/P&L measure.
Personalized insights	Deterministic explanations based on concentration, activity or performance.

Analytics must use actual system state and market data; do not hard-code example numbers.

14. React Full-Stack Dashboard

- Login/Register
- Portfolio overview
- Holdings table
- Paginated trade history
- Buy/Sell form

- P&L dashboard
- Allocation chart
- Sector exposure chart
- Trade-frequency visualization
- Best/worst performers
- Notifications
- Market-price freshness/status



Use React + TypeScript with Recharts or Apache ECharts. Separate API, auth state, chart transformation and presentational components.

15. Power BI / Tableau

Create a BI view using the same underlying database or analytics views used by the application. Include portfolio KPIs, P&L trend, sector exposure, allocation, trade activity and performers. Choose either Power BI or Tableau for the final project rather than implementing both.

16. Security

- Hash passwords using ASP.NET Core Identity or a proven password-hashing mechanism.
- Use short-lived JWT access tokens and validate signature, expiry and claims.
- Enforce ownership on every user-scoped operation.
- Validate all DTOs with FluentValidation or equivalent.
- Keep secrets outside source control.
- Use HTTPS in deployed environments.
- Configure CORS intentionally.
- Apply API rate limiting.
- Never log passwords, JWTs or API keys.
- Return safe error responses.

17. Testing Strategy

Layer	Tools	Examples
Unit	xUnit + Moq	FIFO, validators, analytics, retry decisions
Integration	xUnit + Testcontainers	Real PostgreSQL, Redis, RabbitMQ
API	ASP.NET integration tests	Auth, ownership, validation, pagination, trades
Concurrency	xUnit + Testcontainers	Two simultaneous sells
Messaging	RabbitMQ container	Publish/consume, redelivery, duplicate event
Failure	Integration tests	Broker outage, provider timeout, Redis failure
Frontend	React testing stack	Critical forms and dashboard behavior

Minimum acceptance tests

13. Register/login.
14. User A cannot access User B's data.
15. BUY creates trade, holding and FIFO lot.
16. Partial SELL consumes FIFO lots correctly.
17. Oversell is rejected.
18. Two concurrent sells cannot oversell.
19. Successful trade creates one logical event.
20. Duplicate event does not duplicate notification.
21. Notification retries transient failures.
22. Outbox survives broker outage.
23. Redis cache hit avoids external call.
24. Analytics match independently calculated expected values.
25. Pagination is stable.
26. Rate limiting works.
27. Docker Compose starts the local stack.

18. Docker & Local Environment

The local environment is a core deliverable. A reviewer should be able to clone the repository, configure a small environment file and start the system with one command.

```
docker compose up --build
```

- frontend
- API/gateway
- trade-service
- portfolio-service
- analytics-service
- market-service
- notification-service
- postgres
- redis
- rabbitmq
- prometheus (if used)
- grafana (if used)

Docker requirements

- Multi-stage .NET Dockerfiles.
- Small runtime images.
- Non-root users where practical.
- Health checks.
- Environment-based configuration.
- Persistent PostgreSQL volume.
- Named Docker network.
- Dependency/health startup behavior.
- No secrets baked into images.

19. CI/CD & DevOps

```
push / pull_request

      |

Restore -> Build -> Unit Tests -> Integration Tests

      |

Docker image build -> Security/lint checks

      |

Push to ECR -> Environment-controlled deployment
```

- GitHub Actions is the primary CI pipeline.
- Jenkins may be included as a secondary enterprise-style pipeline demonstration.
- Terraform defines infrastructure.
- Use rollback to a known-good image/task definition.
- Do not duplicate mandatory work across two CI systems without a reason.

20. Observability

Prefer OpenTelemetry for application instrumentation and Prometheus/Grafana for metrics where practical.

Signal	Examples
Logs	trade_id, event_id, correlation_id, service, severity
Metrics	request count, errors, latency, queue depth, retries, cache hits/misses
Traces	API → Trade Service → DB/outbox → RabbitMQ → Notification Service
Health	Database, Redis, RabbitMQ and external-provider readiness

Never log credentials, JWTs, API keys or unnecessary personal data.

21. Terraform & AWS

Component	Role
ECS Fargate	Run containerized services.
RDS PostgreSQL	Managed relational database.
ECR	Container image registry.
S3	Optional artifact/static storage.

CloudWatch	Cloud logs/operational visibility.
Managed Redis	Production cache option; local Redis stays in Docker Compose.
RabbitMQ infrastructure	Managed or separately deployed RabbitMQ-compatible infrastructure beyond local MVP.

Exact cloud topology should respect project budget and time. The requirement is a documented, reproducible deployment path, not a claim that every AWS component is already deployed.

22. Six-Week Implementation Plan

Week	Focus	Deliverables
1	Foundation	Solution/repo, service templates, Docker Compose, PostgreSQL schema/migrations, auth, Instruments, CRUD, Swagger.
2	Trade correctness	Trade Service, transactions, holdings/lots, FIFO, oversell validation, concurrency Testcontainers test.
3	Distributed systems	RabbitMQ, outbox, TradeCompleted, Notification Service, retries, idempotency, DLQ, Redis market data.
4	Analytics + frontend	P&L, allocation, sector exposure, frequency, performers, insights, React dashboard.
5	Production engineering	Docker hardening, GitHub Actions, integration pipeline, OpenTelemetry, Prometheus/Grafana, Terraform, AWS path.
6	Hardening + presentation	Security, rate limiting, versioning, failure tests, rollback docs, BI, README, architecture, demo and interview prep.

23. MVP vs Stretch

Priority	Scope
P0 / MVP	Auth, trade execution, PostgreSQL, holdings, FIFO, concurrency, RabbitMQ, Notification Service, Redis prices, core analytics, React, Docker Compose, tests.
P1	Outbox hardening, DLQ, rate limiting, API versioning, OpenTelemetry, Prometheus/Grafana, BI, GitHub Actions, Terraform.
P2 / Stretch	ECS Fargate deployment, managed Redis/RabbitMQ, advanced alerting, richer personalization, Jenkins, performance testing, blue/green/canary deployment.

Do not sacrifice trade correctness, tests or the complete vertical slice to reach every cloud feature.

24. Recommended Repository Structure

```
TradeFolio/
├── src/
│   ├── Gateway/
│   ├── Services/
```

```
│   │   ├── TradeService/
│   │   ├── PortfolioService/
│   │   ├── AnalyticsService/
│   │   ├── MarketDataService/
│   │   └── NotificationService/
│   ├── BuildingBlocks/
│   │   ├── Contracts/
│   │   ├── Observability/
│   │   └── Common/
│   └── Frontend/tradefolio-web/
├── tests/
│   ├── TradeService.UnitTests/
│   ├── TradeService.IntegrationTests/
│   ├── PortfolioService.UnitTests/
│   ├── AnalyticsService.UnitTests/
│   ├── NotificationService.IntegrationTests/
│   └── EndToEnd/
├── infrastructure/
│   ├── docker/
│   ├── terraform/
│   └── monitoring/
├── docs/
│   ├── architecture/
│   ├── api/
│   ├── database/
│   └── adr/
├── docker-compose.yml
├── .github/workflows/
└── README.md
```

25. Technology Stack

Area	Technology
Backend	ASP.NET Core, current .NET LTS at implementation start; pin SDK in global.json
ORM	Entity Framework Core
Validation	FluentValidation

API docs	Swagger/OpenAPI
Database	PostgreSQL
Cache	Redis
Messaging	RabbitMQ
Frontend	React + TypeScript
Charts	Recharts or Apache ECharts
BI	Power BI or Tableau
Testing	xUnit, Moq, Testcontainers
Containers	Docker, Docker Compose
CI/CD	GitHub Actions; Jenkins optional secondary
IaC	Terraform
Cloud	AWS ECS Fargate, RDS, ECR, S3, CloudWatch
Observability	OpenTelemetry + Prometheus/Grafana

26. Definition of Done / Acceptance Criteria

28. A new user can register and log in.
29. JWT-protected endpoints enforce user isolation.
30. Valid buys and sells work.
31. Concurrent sells cannot oversell and the concurrency test is reproducible.
32. FIFO handles partial sells correctly.
33. Trade completion is propagated asynchronously through RabbitMQ.
34. Notification Service is independently runnable/deployable.
35. Duplicate events are idempotent.
36. Broker outage does not lose a committed event.
37. Market prices use Redis caching.
38. Analytics provide P&L, allocation, sector exposure, trade frequency and performers.
39. React renders the analytics.
40. BI consumes the same data.
41. Docker Compose starts the local stack.
42. Automated tests run in CI.
43. At least two important queries have documented EXPLAIN/EXPLAIN ANALYZE results.
44. Secrets are not committed.
45. Terraform describes the cloud path.
46. Repository contains architecture, setup, API, testing and deployment documentation.

27. Recruiter / Interview Demo Script

47. Start the system with docker compose up --build.
48. Authenticate in React.
49. Show holdings and analytics.
50. Execute a BUY and show holding/lots changing.
51. Execute a partial SELL and explain FIFO cost basis.
52. Run the concurrent-sell test and show overselling is prevented.
53. Show TradeCompleted through RabbitMQ.
54. Show Notification Service consuming the event.
55. Redeliver the event and demonstrate idempotency.
56. Stop RabbitMQ, complete a trade, show pending outbox, restart RabbitMQ and show eventual publication.
57. Show Redis market-price caching.
58. Show P&L, allocation, sector and frequency analytics.
59. Open the BI dashboard.
60. Show GitHub Actions build/test pipeline.

61. Show containers and service health.
62. Show Terraform/AWS architecture.
63. Show a trace/metric for the end-to-end trade flow.

28. Interview Topics

- Why microservices instead of a monolith?
- Where are service boundaries and data ownership?
- How do you prevent simultaneous overselling?
- Why Read Committed + row locking?
- Why the transactional outbox?
- What happens when RabbitMQ is down?
- How is the consumer idempotent?
- Why Redis and what if it fails?
- How were PostgreSQL indexes chosen?
- What did EXPLAIN ANALYZE show?
- Why FIFO and what is its complexity?
- How does JWT authentication work?
- How is user data isolated?
- How does Docker networking work?
- How would you scale a service independently?
- How do you trace a request across services?
- How does CI/CD deploy containers?
- How do you roll back?
- When would you choose a monolith?

29. Risks & Mitigations

Risk	Mitigation
Scope too large	Prioritize P0 vertical slice; defer advanced AWS/BI/observability.
Microservices complexity	Use only meaningful boundaries and Docker Compose.
Oversell race	Explicit locking + transaction + real concurrency test.
Lost events	Transactional outbox + retrying publisher.
Duplicate notifications	Unique event IDs + idempotent consumer.
Market API outage	Redis cache + stale indication + provider abstraction.
Database performance	Indexes + EXPLAIN ANALYZE.
Secret leakage	Environment/secret management and scanning.
Cloud cost	Keep AWS deployment optional/P2 and destroy unused resources.
Analytics mismatch	Explicit accounting rules + independent calculation tests.

30. First 10 Build Actions

64. Create repository and solution structure.
65. Pin the .NET SDK version in global.json using the current LTS available at implementation start.
66. Create service templates and shared contract/common projects.
67. Create Docker Compose with PostgreSQL, Redis and RabbitMQ.
68. Implement authentication/JWT and ownership authorization.
69. Create migrations for Users, Instruments, Trades, Holdings, HoldingLots and OutboxMessages.
70. Implement the first end-to-end BUY flow.

71. Implement FIFO lots and partial SELL.
72. Write the Testcontainers concurrency test.
73. Add RabbitMQ, outbox and Notification Service and demonstrate the first TradeCompleted event.

31. Final Deliverables

- Git repository and complete source.
- Architecture diagram.
- Docker Compose environment.
- ASP.NET Core microservices.
- PostgreSQL schema/migrations.
- Redis market-data caching.
- RabbitMQ event contracts/topology.
- Independent Notification Service.
- FIFO accounting.
- Concurrency integration test.
- REST API + Swagger/OpenAPI.
- React dashboard.
- Power BI or Tableau dashboard.
- xUnit/Moq/Testcontainers suite.
- GitHub Actions workflow.
- Jenkins pipeline example if included.
- Terraform definitions.
- AWS deployment documentation.
- Observability configuration.
- README with one-command setup.
- Architecture decision/tradeoff records.
- Final demo script.

32. Final Portfolio Narrative

TradeFolio should be presented as a full-stack, production-style microservices project rather than merely a stock dashboard. The analytics experience gives the product business value, while the architecture demonstrates transactional trade execution, concurrency control, FIFO accounting, event-driven notifications, idempotent consumers, Redis caching, REST APIs, React, Docker, automated testing, CI/CD, Terraform, AWS and observability.

The strongest evidence is a few hard problems demonstrated end-to-end: two simultaneous sells cannot oversell; a committed trade is not lost when RabbitMQ is temporarily unavailable; duplicate events do not create duplicate notifications; market data is cached; analytics reflect actual portfolio state; the complete system runs locally in containers; and the same services can be packaged for cloud deployment.